Product Store

Component Design Document

1 Description

The product store saves a predefined set of data products from the database to two byte arrays (memory regions) provided at initialization, usually located in nonvolatile storage (i.e. MRAM). The set of data products to save is configured via an autocoded table provided at instantiation, produced from a stored_products.yaml model file named <assembly_name>.stored_products.yaml, or <specific_name>.<assembly_name>.stored_products.yaml when more than one product store is instantiated within the same assembly. Data products are saved upon receipt of a tick (which can be divided down and enabled/disabled by command) or by command, and can be restored back into the data product database by command or at Set_Up. The store is double buffered so that a reboot in the middle of a save can never corrupt the only good copy - each save writes the copy NOT holding the most recent valid save, stamping it with a monotonic save counter and writing its CRC last, and a restore reads from the valid copy holding the newest counter. A reboot mid-save therefore costs at most one save interval of freshness. Each copy is protected by a CRC which is written on every save and checked prior to every restore, protecting against the restoration of corrupted or never-initialized memory contents. Each store entry also holds the stored length of its data product, where a length of zero marks an entry that has never been saved. Entries whose data product cannot be fetched during a save keep the values from the most recent valid save, and entries that have never been saved are silently skipped on restore, leaving those data products unavailable in the database rather than restoring meaningless values. A command is provided to dump the current contents of both store copies, each into its own packet. The autocoder limits the size of each store copy to fit within a single Packet.T, and requires every stored data product to have an always valid type, so a restore can never produce a value that fails validation.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 8
- **Number of Invokee Connectors** - 2
- **Number of Invoker Connectors** - 6
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*

- **Number of Commands** - 5
- **Number of Parameters** - *None*
- **Number of Events** - 12
- **Number of Faults** - *None*
- **Number of Data Products** - 3
- **Number of Data Dependencies** - *None*
- **Number of Packets** - 2

3.2 Diagram

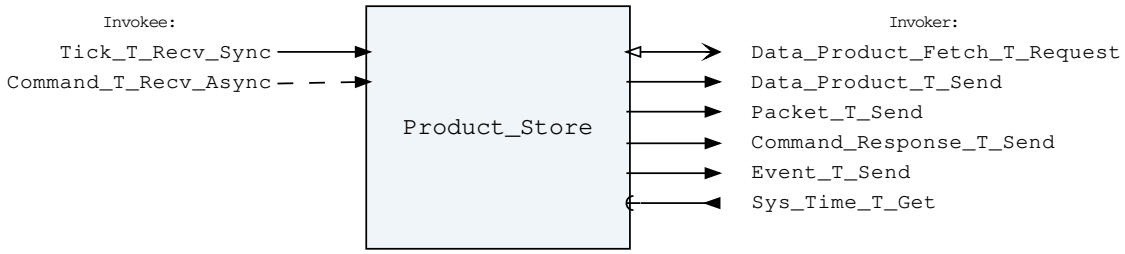

Figure 1: Product Store component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Product Store Invokee Connectors

Name	Kind	Type	Return_Type	Count
Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Command_T_Recv_Async	recv_async	Command.T	-	1

Connector Descriptions:

- **Tick_T_Recv_Sync** - This is the base tick for the component. Commands are dispatched from the queue on every tick, and the data products are saved to the store every Ticks_Per_Save ticks (when saving on tick is enabled).
- **Command_T_Recv_Async** - This is the command receive connector.

3.3.2 Internal Queue

This component contains an internal first-in-first-out (FIFO) queue to handle asynchronous messages. This queue is sized at initialization as a configurable number of bytes. Determining the size of the component queue can be difficult. The following table lists the connectors that will put asynchronous messages onto the queue, and the maximum sizes of each of those messages on the queue. Note that each message put onto the queue also incurs an overhead on the queue of 5 additional bytes, which is included in the max message size below:

Table 2: Product Store Asynchronous Connectors

Name	Type	Max Size (bytes)
Command_T_Recv_Async	Command.T	265

If you are unsure how to size the queue of this component, it is recommended that you make the queue size a multiple of the largest size found above.

3.3.3 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 3: Product Store Invoker Connectors

Name	Kind	Type	Return_Type	Count
Data_Product_ Fetch_T_Request	request	Data_Product_ Fetch.T	Data_Product_ Return.T	1
Data_Product_T_ Send	send	Data_Product.T	-	1
Packet_T_Send	send	Packet.T	-	1
Command_ Response_T_Send	send	Command_ Response.T	-	1
Event_T_Send	send	Event.T	-	1
Sys_Time_T_Get	get	-	Sys_Time.T	1

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database for saving.
- **Data_Product_T_Send** - Data products are sent out of this connector, both when restoring the store contents into the data product database and when reporting this component's own counter data products.
- **Packet_T_Send** - Send a packet holding the contents of the store when a dump is commanded.
- **Command_Response_T_Send** - This connector is used to register and respond to the component's commands.
- **Event_T_Send** - Events are sent out of this connector.
- **Sys_Time_T_Get** - The system time is retrieved via this connector.

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component requires a description of the data products it is responsible for saving and restoring. This description should be provided as an autocoded output from a stored_products.yaml model file. This component contains the following instantiation parameters in its discriminant:

Table 4: Product Store Instantiation Parameters

Name	Type
Store_Description	Product_Store_Types.Store_Description_Access_Type

Parameter Descriptions:

- **Store_Description** - The description of the data product store to manage.

3.5.2 Component Base Initialization

This component achieves base class initialization using the `init_Base` subprogram. This subprogram requires the following parameters:

Table 5: Product Store Base Initialization Parameters

Name	Type
Queue_Size	Natural

Parameter Descriptions:

- **Queue_Size** - The number of bytes that can be stored in the component's internal queue.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 6: Product Store Set Id Bases Parameters

Name	Type
Command_Id_Base	Command_Types.Command_Id_Base
Data_Product_Id_Base	Data_Product_Types.Data_Product_Id_Base
Event_Id_Base	Event_Types.Event_Id_Base
Packet_Id_Base	Packet_Types.Packet_Id_Base

Parameter Descriptions:

- **Command_Id_Base** - The value at which the component's command identifiers begin.
- **Data_Product_Id_Base** - The value at which the component's data product identifiers begin.
- **Event_Id_Base** - The value at which the component's event identifiers begin.
- **Packet_Id_Base** - The value at which the component's unresolved packet identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains no data dependencies.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. The component is initialized by providing the two memory regions it is to manage, which hold the two copies of the data product store. The `init` subprogram requires the following parameters:

Table 7: Product Store Implementation Initialization Parameters

Name	Type	Default Value
Bytes_A	Basic_Types.Byte_Array_Access	*None provided*
Bytes_B	Basic_Types.Byte_Array_Access	*None provided*
Restore_On_Set_Up	Boolean	False
Ticks_Per_Save	Positive	1
Commands_Dispatched_Per_Tick	Positive	3

Parameter Descriptions:

- **Bytes_A** - A pointer to an allocation of bytes to be used for storing copy A of the data products. The size of this byte array MUST be at least Store_Description.Store_Size bytes in length (which includes the CRC, save counter, and save time header). Only the first Store_Description.Store_Size bytes will be used by this component. This allocation must not overlap the allocation provided for Bytes_B. The two copies may be placed in different memory banks so that a fault contained to one bank cannot corrupt both copies.
- **Bytes_B** - A pointer to an allocation of bytes to be used for storing copy B of the data products, with the same size requirement as Bytes_A. This allocation must not overlap the allocation provided for Bytes_A.
- **Restore_On_Set_Up** - If set to True, the component will attempt to restore the stored data products into the data product database during Set_Up, seeding the database with the values saved before the last reboot. If neither store copy holds a valid CRC (i.e. the store was never written or was corrupted) the restore is skipped and error events are produced.
- **Ticks_Per_Save** - The number of ticks that must be received before the data products are saved to the store. This allows the component to be connected to a rate group running at a speed appropriate for command responsiveness (i.e. 1 Hz), while saving to the store at a slower rate (i.e. every 600 ticks for a 10 minute save cadence).
- **Commands_Dispatched_Per_Tick** - The number of commands executed per tick, if any are in the queue.

3.6 Commands

These are the commands for the Product Store component.

Table 8: Product Store Commands

Local ID	Command Name	Argument Type
0	Save_Products	-
1	Restore_Products	-
2	Dump_Store	-
3	Enable_Save_On_Tick	-
4	Disable_Save_On_Tick	-

Command Descriptions:

- **Save_Products** - Save the configured data products from the database into the store. This works regardless of whether saving on tick is enabled or disabled. The current time is used as the save time, even if the store is configured for Tick_Time, since no tick is available.
- **Restore_Products** - Restore the data product values held in the store back into the data product database. The CRC of each store copy is checked prior to the restore, and the values are restored from the valid copy holding the newest save counter. The command fails if neither copy's CRC validates.

- **Dump_Store** - Dump the current contents of both store copies, each into its own packet. The store contents are dumped as-is, without validating the CRCs, so that corrupted store contents can be inspected on the ground.
- **Enable_Save_On_Tick** - Enable the automatic saving of the data products to the store upon receipt of a tick. Saving on tick is enabled by default at startup.
- **Disable_Save_On_Tick** - Disable the automatic saving of the data products to the store upon receipt of a tick. This can be used to protect the store contents when bad data is suspected of feeding the store. Saves can still be performed by command while disabled.

3.7 Parameters

The Product Store component has no parameters.

3.8 Events

Events for the Product Store component.

Table 9: Product Store Events

Local ID	Event Name	Parameter Type
0	Products_Saved	Store_Copy_Info.T
1	Products_Restored	Store_Copy_Info.T
2	Store_Dumped	-
3	Save_On_Tick_Enabled	-
4	Save_On_Tick_Disabled	-
5	Data_Product_Missing_On_Save	Data_Product_Id.T
6	Data_Product_Id_Out_Of_Range	Data_Product_Id.T
7	Data_Product_Length_Mismatch	Invalid_Data_Product_Length.T
8	Stored_Length_Mismatch	Invalid_Stored_Length.T
9	Store_Crc_Invalid	Copy_Crc_Mismatch_Info.T
10	Invalid_Command_Received	Invalid_Command_Info.T
11	Dropped_Command	Command_Header.T

Event Descriptions:

- **Products_Saved** - The data products were saved to the store by command. The parameter reports the store copy that was written and the save counter it now holds.
- **Products_Restored** - The data products held in the store were restored into the data product database. The parameter reports the store copy that was restored from (the valid copy holding the newest save counter) and the save counter it holds.
- **Store_Dumped** - Produced packets with the contents of both copies of the store.
- **Save_On_Tick_Enabled** - The automatic saving of data products to the store upon receipt of a tick was enabled by command.
- **Save_On_Tick_Disabled** - The automatic saving of data products to the store upon receipt of a tick was disabled by command.
- **Data_Product_Missing_On_Save** - A data product was not available from the database when fetched for saving, so its slot in the store was left unchanged, preserving the last saved value (or the never-saved marker if no value was ever saved). This event can be disabled per data product in the stored products model.
- **Data_Product_Id_Out_Of_Range** - A data product id was reported as out of range by the database when fetched for saving, so its slot in the store was left unchanged. This indicates a misconfiguration between the stored products model and the data product database.

- **Data_Product_Length_Mismatch** - A data product was fetched for saving but contained an unexpected length, so its slot in the store was left unchanged.
- **Stored_Length_Mismatch** - A store entry held a stored length that is neither zero (never saved) nor the expected length for the data product, so the entry was not restored. This indicates that the stored products model has changed since the store was last written.
- **Store_Crc_Invalid** - A restore found no store copy with a valid CRC, so the restore was not performed. This event is produced once per copy, reporting that copy's CRC mismatch. This is expected on the first boot before the store has ever been written.
- **Invalid_Command_Received** - A command was received with invalid parameters.
- **Dropped_Command** - A command was dropped due to a full queue.

3.9 Data Products

Data products for the Product Store component.

Table 10: Product Store Data Products

Local ID	Data Product Name	Type
0x0000 (0)	Save_Count	Packed_U16.T
0x0001 (1)	Restore_Count	Packed_U16.T
0x0002 (2)	Crc_Invalid_Count	Packed_U16.T

Data Product Descriptions:

- **Save_Count** - The number of times the data products have been saved to the store, either by tick or by command. This counter rolls over.
- **Restore_Count** - The number of times the store contents have been successfully restored into the data product database. This counter rolls over.
- **Crc_Invalid_Count** - The number of times a restore was refused because neither store copy held a valid CRC. This counter rolls over.

3.10 Data Dependencies

The Product Store component has no data dependencies.

3.11 Packets

Packets for the Product Store component. The contents of these packets are populated based on the stored products model provided to the Product Store component at instantiation. The store is double buffered, and each copy is dumped in its own packet.

Table 11: Product Store Packets

Local ID	Packet Name	Type
0x0000 (0)	Stored_Products_A	*Undefined*
0x0001 (1)	Stored_Products_B	*Undefined*

Packet Descriptions:

- **Stored_Products_A** - This packet contains the contents of copy A of the data product store managed by this component, including the store CRC, the save counter, the save time, and each stored data product (with its timestamp, if configured).

- **Stored_Products_B** - This packet contains the contents of copy B of the data product store managed by this component, including the store CRC, the save counter, the save time, and each stored data product (with its timestamp, if configured).

3.12 Faults

The Product Store component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Product_Store_Tests* Test Suite

This is a unit test suite for the Product Store component.

Test Descriptions:

- **Test_Set_Up_Seeding** - This unit test tests that the counter data products are seeded with zero at Set_Up.
- **Test_Nominal_Save** - This unit test tests saving the data products to the store upon receipt of a tick.
- **Test_Nominal_Restore** - This unit test tests restoring the data products from the store by command, including all of the restore timestamp modes.
- **Test_Restore_On_Set_Up** - This unit test tests restoring the data products from the store at Set_Up, both with a valid and an invalid store.
- **Test_Crc_Invalid_On_Restore** - This unit test tests the component's response to a restore command when the store contents are corrupted.
- **Test_Save_Command** - This unit test tests saving the data products to the store by command.
- **Test_Save_On_Tick_Enable_Disable** - This unit test tests disabling and enabling the automatic saving of data products on tick by command.
- **Test_Ticks_Per_Save** - This unit test tests the tick divider, which saves the data products only every Ticks_Per_Save ticks.
- **Test_Missing_Data_Product** - This unit test tests the component's response to a data product that is missing from the database on save, verifying that slots keep the values from the most recent valid save and that never-saved slots restore nothing.
- **Test_Id_Out_Of_Range** - This unit test tests the component's response to a data product id reported as out of range by the database on save, which produces an unmaskable event.
- **Test_Length_Mismatch** - This unit test tests the component's response to a fetched data product with an unexpected length.
- **Test_Dump_Store** - This unit test tests dumping the contents of both store copies into packets by command.
- **Test_Mid_Save_Reboot_Recovery** - This unit test tests that a reboot in the middle of a save (simulated by corrupting the copy that the next save would write) costs only one save of freshness, with the restore falling back to the intact copy and subsequent saves recovering the corrupted copy.
- **Test_Offset_Memory_Regions** - This unit test tests that the component operates correctly when given byte arrays that are larger than the store size and that have nonzero first indices, verifying that only the first Store_Size bytes of each allocation are used.

- **Test_Invalid_Command** - This unit test tests the component's response to an invalid command.
- **Test_Full_Queue** - This unit test tests a command being dropped due to a full queue.
- **Test_Restore_Skips_Unwritten** - This unit test tests that a restore silently skips store entries that have never been saved, leaving those data products unavailable in the database.
- **Test_Stored_Length_Mismatch** - This unit test tests that a restore refuses a store entry whose stored length does not match the expected length, which indicates the stored products model has changed since the store was written.

4.2 *Product_Store_Tests* Test Suite

This is a unit test suite for the Product Store component with a store configured to save the current time.

Test Descriptions:

- **Test_Save_Current_Time** - This unit test tests saving the data products to a store configured to save the current time upon receipt of a tick, and restoring them with the save time.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Command.T:

Generic command packet for holding arbitrary commands

Table 12: Command Packed Record : 2080 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Command_ Header.T	-	40	0	39	-
Arg_Buffer	Command_ Types. Command_Arg_ Buffer_Type	-	2040	40	2079	Header.Arg_ Buffer_Length

Field Descriptions:

- **Header** - The command header
- **Arg_Buffer** - A buffer that contains the command arguments

Command_Header.T:

Generic command header for holding arbitrary commands

Table 13: Command_Header Packed Record : 40 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Source_Id	Command_Types.Command_Source_Id	0 to 65535	16	0	15
Id	Command_Types.Command_Id	0 to 65535	16	16	31
Arg_Buffer_Length	Command_Types.Command_Arg_Buffer_Length_Type	0 to 255	8	32	39

Field Descriptions:

- **Source_Id** - The source ID. An ID assigned to a command sending component.
- **Id** - The command identifier
- **Arg_Buffer_Length** - The number of bytes used in the command argument buffer

Command_Response.T:

Record for holding command response data.

Table 14: Command_Response Packed Record : 56 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Source_Id	Command_Types.Command_Source_Id	0 to 65535	16	0	15
Registration_Id	Command_Types.Command_Registration_Id	0 to 65535	16	16	31
Command_Id	Command_Types.Command_Id	0 to 65535	16	32	47
Status	Command_Enums.Command_Response_Status.E	0 => Success 1 => Failure 2 => Id_Error 3 => Validation_Error 4 => Length_Error 5 => Dropped 6 => Register 7 => Register_Source	8	48	55

Field Descriptions:

- **Source_Id** - The source ID. An ID assigned to a command sending component.
- **Registration_Id** - The registration ID. An ID assigned to each registered component at initialization.
- **Command_Id** - The command ID for the command response.
- **Status** - The command execution status.

Copy_Crc_Mismatch_Info.T:

A packed record which holds a computed CRC alongside the CRC that was expected for one copy of the double-buffered data product store, used for reporting a CRC mismatch on that copy.

Table 15: Copy_Crc_Mismatch_Info Packed Record : 40 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Copy	Product_Store_ Enums.Store_Copy.E	0 => Copy_A 1 => Copy_B	8	0	7
Computed_Crc	Crc_16.Crc_16_Type	-	16	8	23
Expected_Crc	Crc_16.Crc_16_Type	-	16	24	39

Field Descriptions:

- **Copy** - The store copy whose CRC did not validate.
- **Computed_Crc** - The CRC computed over the contents of the store copy.
- **Expected_Crc** - The CRC held in the store copy's header, which the computed CRC was expected to match.

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 16: Data_Product Packed Record : 344 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_ Header.T	-	88	0	87	-
Buffer	Data_Product_ Types.Data_ Product_ Buffer_Type	-	256	88	343	Header.Buffer_ Length

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 17: Data_Product_Fetch Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The data product identifier

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 18: Data_Product_Header Packed Record : 88 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	64	0	63
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	64	79
Buffer_Length	Data_Product_ Types.Data_Product_ Buffer_Length_Type	0 to 32	8	80	87

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Data_Product_Id.T:

A packed record which holds a data product identifier.

Table 19: Data_Product_Id Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The data product identifier

Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 20: Data_Product_Return Packed Record : 352 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
The_ Status	Data_ Product_ Enums. Fetch_ Status.E	0 => Success 1 => Not_Available 2 => Id_Out_Of_Range	8	0	7	-
The_Data_ Product	Data_ Product.T	-	344	8	351	-

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

Event.T:

Generic event packet for holding arbitrary events

Table 21: Event Packed Record : 344 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Event_Header.T	-	88	0	87	-
Param_Buffer	Event_Types. Parameter_ Buffer_Type	-	256	88	343	Header.Param_ Buffer_Length

Field Descriptions:

- **Header** - The event header
- **Param_Buffer** - A buffer that contains the event parameters

Event_Header.T:

Generic event packet for holding arbitrary events

Table 22: Event_Header Packed Record : 88 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	64	0	63
Id	Event_Types.Event_ Id	0 to 65535	16	64	79
Param_Buffer_Length	Event_Types. Parameter_Buffer_ Length_Type	0 to 32	8	80	87

Field Descriptions:

- **Time** - The timestamp for the event.
- **Id** - The event identifier
- **Param_Buffer_Length** - The number of bytes used in the param buffer

Invalid_Command_Info.T:

Record for holding information about an invalid command

Table 23: Invalid_Command_Info Packed Record : 112 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Command_Types. Command_Id	0 to 65535	16	0	15
Errant_Field_ Number	Interfaces. Unsigned_32	0 to 4294967295	32	16	47
Errant_Field	Basic_Types.Poly_ Type	-	64	48	111

Field Descriptions:

- **Id** - The command Id received.
- **Errant_Field_Number** - The field that was invalid. 1 is the first field, 0 means unknown field, 2**32 means that the length field of the command was invalid.

- **Errant_Field** - A polymorphic type containing the bad field data, or length when Errant_Field_Number is 2**32.

Invalid_Data_Product_Length.T:

A packed record which holds data related to an invalid data product length.

Table 24: Invalid_Data_Product_Length Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Header	Data_Product_Header.T	-	88	0	87
Expected_Length	Data_Product_Types.Data_Product_Buffer_Length_Type	0 to 32	8	88	95

Field Descriptions:

- **Header** - The packet identifier
- **Expected_Length** - The packet length bound that the length failed to meet.

Invalid_Stored_Length.T:

A packed record which holds data related to an invalid stored data product length found in the store on restore.

Table 25: Invalid_Stored_Length Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types.Data_Product_Id	0 to 65535	16	0	15
Stored_Length	Interfaces.Unsigned_8	0 to 255	8	16	23
Expected_Length	Data_Product_Types.Data_Product_Buffer_Length_Type	0 to 32	8	24	31

Field Descriptions:

- **Id** - The data product identifier of the store entry.
- **Stored_Length** - The length found in the store for this entry. A value that is neither zero (never saved) nor the expected length indicates that the stored products model has changed since the store was written.
- **Expected_Length** - The length that the stored length was expected to match.

Packed_U16.T:

Single component record for holding packed unsigned 16-bit value.

Table 26: Packed_U16 Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Interfaces.Unsigned_16	0 to 65535	16	0	15

Field Descriptions:

- **`Value`** - The 16-bit unsigned integer.

Packet.T:

Generic packet for holding arbitrary data

Table 27: Packet Packed Record : 10080 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Packet_ Header.T	-	112	0	111	-
Buffer	Packet_ Types.Packet_ Buffer_Type	-	9968	112	10079	Header. Buffer_Length

Field Descriptions:

- **`Header`** - The packet header
- **`Buffer`** - A buffer that contains the packet data

Packet_Header.T:

Generic packet header for holding arbitrary data

Table 28: Packet_Header Packed Record : 112 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	64	0	63
Id	Packet_Types. Packet_Id	0 to 65535	16	64	79
Sequence_Count	Packet_Types. Sequence_Count_Mod_ Type	0 to 16383	16	80	95
Buffer_Length	Packet_Types. Packet_Buffer_ Length_Type	0 to 1246	16	96	111

Field Descriptions:

- **`Time`** - The timestamp for the packet item.
- **`Id`** - The packet identifier
- **`Sequence_Count`** - Packet Sequence Count
- **`Buffer_Length`** - The number of bytes used in the packet buffer

Store_Copy_Info.T:

A packed record identifying a copy of the double-buffered data product store and the save counter it holds, used for reporting the copy written by a save or read by a restore.

Table 29: Store_Copy_Info Packed Record : 40 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Copy	Product_Store_ Enums.Store_Copy. E	0 => Copy_A 1 => Copy_B	8	0	7
Save_Counter	Interfaces. Unsigned_32	0 to 4294967295	32	8	39

Field Descriptions:

- **Copy** - The store copy that was written or read.
- **Save_Counter** - The monotonic save counter held in the store copy. The copy holding the larger counter holds the more recent save.

Sys_Time.T:

A record which holds a time stamp using GPS format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 30: Sys_Time Packed Record : 64 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces. Unsigned_32	0 to 4294967295	32	32	63

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^32) sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 31: Tick Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	64	0	63
Count	Interfaces. Unsigned_32	0 to 4294967295	32	64	95

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Command_Enums.Command_Response_Status.E:

This status enumeration provides information on the success/failure of a command through the command response connector.

Table 32: Command_Response_Status Literals:

Name	Value	Description
Success	0	Command was passed to the handler and successfully executed.
Failure	1	Command was passed to the handler not successfully executed.
Id_Error	2	Command id was not valid.
Validation_Error	3	Command parameters were not successfully validated.
Length_Error	4	Command length was not correct.
Dropped	5	Command overflowed a component queue and was dropped.
Register	6	This status is used to register a command with the command routing system.
Register_Source	7	This status is used to register command sender's source id with the command router for command response forwarding.

Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 33: Fetch_Status Literals:

Name	Value	Description
Success	0	The data product was returned successfully.
Not_Available	1	No data product is yet available for the provided id.
Id_Out_Of_Range	2	The data product id was out of range.

Product_Store_Enums.Store_Copy.E:

Identifies one of the two copies of the double-buffered data product store.

Table 34: Store_Copy Literals:

Name	Value	Description
Copy_A	0	The first copy of the store.
Copy_B	1	The second copy of the store.